A CONFIGURABLE PERFORMANCE MONITORING AND ANALYTICS IP FOR RISC-V PROCESSORS

INNOVATIVE PROJECT Nov/Dec 2026

INNOVATIVE / MULTI-DISCIPLINARY PROJECT

REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF **BACHELOR OF ENGINEERING** IN **ELECTRONICS AND COMMUNICATION ENGINEERING** OF THE ANNA UNIVERSITY

PROJECT WORK

BATCH 2024 – 2028

Submitted by

LAKSANA A – 722824106077

PRANAV J - 722824106117

PRAVIN A – 722824106121

THRISHNA M – 722824106175

Under the Guidance of

Dr. M. Saravanan, M.E., Ph.D.,

Assistant Professor

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

Sri Eshwar College of Engineering

(An Autonomous Institution)

Kinathukadavu (Tk), Coimbatore - 641 202, Tamil Nadu

Approved by AICTE, New Delhi and Affiliated to Anna University, Chennai

A CONFIGURABLE PERFORMANCE MONITORING AND ANALYTICS IP FOR RISC-V PROCESSORS

INNOVATIVE PROJECT Nov/Dec 2026

INNOVATIVE / MULTI-DISCIPLINARY PROJECT

REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF **BACHELOR OF ENGINEERING** IN **ELECTRONICS AND COMMUNICATION ENGINEERING** OF THE ANNA UNIVERSITY

PROJECT WORK

Submitted by

LAKSANA A – 722824106077

PRANAV J - 722824106117

PRAVIN A – 722824106121

THRISHNA M – 722824106175

BATCH 2024 – 2028

Under the Guidance of

Dr. M. Saravanan, M.E., Ph.D.,

Assistant Professor

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

Sri Eshwar College of Engineering

(An Autonomous Institution)

Kinathukadavu (Tk), Coimbatore - 641 202, Tamil Nadu

Approved by AICTE, New Delhi and Affiliated to Anna University, Chennai



BONAFIDE CERTIFICATE

Certified that this Report titled **"A CONFIGURABLE PERFORMANCE MONITORING AND ANALYTICS IP FOR RISC-V PROCESSORS"** is the bonafide work of

LAKSANA A	**722824106077**
PRANAV J	**722824106117**
PRAVIN A	**722824106121**
THRISHNA M	**722824106175**

who carried out the project work under my supervision.

---------- .	– ----------
SIGNATURE	**SIGNATURE**
Dr. N. Shanmugasundaram, M.E., Ph.D., HEAD OF THE DEPARTMENT, Department of ECE, Sri Eshwar College of Engineering, Coimbatore – 641 202.	**Dr. M. Saravanan, M.E., Ph.D.,** **SUPERVISOR** ASSISTANT PROFESSOR, Department of ECE, Sri Eshwar College of Engineering, Coimbatore – 641 202.
Submitted for the **Autonomous Semester End Innovative / Multi-Disciplinary Project Viva-Voce** held on …….	
INTERNAL EXAMINER	**EXTERNAL EXAMINER**

QUALITY POLICY

To establish a system of Quality Enhancement, which would on a continuous basis evaluate and enhance the quality of teaching – learning, research and extension activities of the institution, leading to improvements in all processes, enabling the institution to attain excellence.

INSTITUTE VISION

To be recognized as a premier institution, grooming students into globally acknowledged engineering professionals.

INSTITUTE MISSION

- Providing outcome and value-based engineering education
- Nurturing research and entrepreneurial culture
- Enabling students to be industry ready and fulfill their career aspirations
- Grooming students through behavioral and leadership training programs
- Making students socially responsible

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

DEPARTMENT VISION

"To groom students into futuristic and globally competent Electronics and Communication engineering professionals."

DEPARTMENT MISSION

To impart quality education with moral and ethical values to develop competent engineers, leaders and successful entrepreneurs

To establish state-of-art infrastructure and provide opportunities to update on emerging tools and technologies

To empower the faculty towards excellence in teaching – learning, consultancy, research and development activities

To foster socially relevant and industry oriented innovation among students.

PROGRAM EDUCATIONAL OBJECTIVES

PEO1: Graduates will pursue careers in Electronic System Design, Internet of Things, System on Chip, and Networking domains related to the design and manufacture of electronic systems, providing solutions for industrial and societal problems.

PEO2: The Graduate will engage in post-graduate studies in the field of Electronics and Communication Engineering and management sciences leading to academic and research careers.

PEO3: Graduates will take up entrepreneurship as a career.

PROGRAM OUTCOMES

PO1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using the first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions for complex problems.

PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture, and environment.

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8: Individual and Collaborative Teamwork: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES

PSO1. Interpret and Design Electronic systems using the Internet of Things, VLSI Technology and Efficient signal processing algorithms.

PSO2. Apply knowledge to solve problems in Communication Systems and Networks.

ACKNOWLEDGEMENT

The success of a work depends on a team and cooperation. We take this opportunity to express our gratitude and thanks to everyone who helped us in our project. We would like to thank the management for excellent facilities and the constant support provided by them to complete this project. It is indeed our great honor bounded duty to thank our beloved Chairman **Mr. R. Mohanram,** for his academic interest shown towards the students. We are indebted to our Director **Mr. R. Rajaram,** for motivating and providing us with all facilities. We wish to express our sincere regards and deep sense of gratitude to **Dr. Sudha Mohanram, M.E, Ph.D.,** Principal, for the excellent facilities and encouragement provided during the course of the study and project.

We are indebted to **Dr. N. Shanmugasundaram, M.E., Ph.D.,** Professor & Head of Electronics and Communication Department for having permitted us to carry out this project and for giving the complete freedom to utilize the resources of the department.

We express our sincere thanks to **Dr. N. Kumareshan, M.E., PhD.,** Associate Professor/ECE, our project coordinator for her valuable support and encouragement given to us for this project.

We express our sincere thanks to **Dr. M. Saravanan, M.E., Ph.D.,** Assistant Professor/ECE, our project guide for his motivation and valuable guidance given to us in completing this project.

We solemnly express our thanks to all the teaching and non-teaching staff of the Electronics and Communication Engineering Department, family and friends for their valuable support which inspired us to work on this project.

ABSTRACT

Modern RISC-V processors execute instructions through a sequence of pipeline stages to improve instruction throughput and overall processor performance. However, evaluating the efficiency of a pipelined processor requires more than observing the final program output. Processor designers must understand the number of instructions executed, total clock cycles, data dependencies, pipeline stalls, control hazards, forwarding operations, and the utilization of individual pipeline stages. Conventional performance monitoring approaches often provide raw event counts that require additional waveform inspection or external software-based processing to obtain meaningful performance information. This project presents the design and simulation of a Configurable Performance Monitoring and Analytics Intellectual Property (IP) for RISC-V processors. The proposed IP is designed to observe processor execution events without modifying the functional logic of the host processor. A wrapper interface collects the required pipeline signals and transfers them to the analytics module for event counting and performance evaluation. The architecture consists of five major components: Event Collector, Event Counter Bank, Analytics Engine, Mode Controller, and Register Bank. The proposed system supports two operating modes: Architectural Profiling and Microarchitectural Analysis. Architectural Profiling focuses on instruction execution, clock cycles, branches, and memory operations. Microarchitectural Analysis evaluates pipeline behavior, including data hazards, load-use hazards, control hazards, stalls, flushes, forwarding activity, and pipeline-stage utilization. The Analytics Engine calculates important performance indicators such as Instructions Per Cycle (IPC), Cycles Per Instruction (CPI), pipeline efficiency, stage utilization, hazard distribution, memory operation ratio, and forwarding efficiency. The calculated results are stored in a Register Bank for systematic retrieval and analysis. The design is evaluated through RTL simulation using a 32-bit, three-stage pipelined RV32I processor and instruction workloads of approximately 2K and 10K instructions. The results reported in the project paper show an IPC of 0.92, a CPI of 1.07–1.08, approximately 92% pipeline efficiency, and 100% forwarding efficiency for the evaluated workloads.

Keywords: RISC-V, Performance Monitoring Unit, Analytics IP, RV32I, RTL Simulation, IPC, CPI, Pipeline Hazards, Forwarding Efficiency, Microarchitectural Analysis.

TABLE OF CONTENTS

CHAPTER No.	TITLE	PAGE No.
	ACKNOWLEDGEMENT	**1**
	ABSTRACT	**2**
	LIST OF FIGURES	**5**
	LIST OF TABLES	**6**
	LIST OF ABBREVIATIONS	**7**
1	**INTRODUCTION**	**8**
	1.1 Background and Context of RISC-V Processors	8
	1.2 Project Motivation and Problem Statement	8
	1.3 Project Objectives and Success Criteria	9
	1.4 Existing Approaches and Comparative Analysis	10
	1.5 Scope and Methodology Overview	11
2	**LITERATURE SURVEY**	**13**
	2.1 RISC-V Performance Monitoring and Existing Approaches	13
	2.2 Processor Performance Metrics and Event Data Models	13
	2.3 Research Gaps and Motivation for the Proposed System	15
3	**SYSTEM OVERVIEW**	**16**
	3.1 Overall System Architecture	16
	3.2 Design Technology and Tool Selection	17
	3.3 Performance Register Organization and Data Management	18
4	**PROCESSOR EVENT SOURCES AND INTEGRATION ARCHITECTURE**	**19**
	4.1 Processor Pipeline Signal Integration	19
	4.2 Event Classification and Signal Mapping	19
	4.3 Event Validation and Counter Update Logic	20
	4.4 Handling Reset, Stalls, Flushes, and Execution Completion	21
5	**SOFTWARE AND RTL DESIGN COMPONENTS**	**22**
	5.1 Hardware Description Languages	22
	5.2 Simulation and Verification Environment	22
	5.3 RTL Design Modules	23
	5.4 Development and Debugging Tools	24
6	**METHODOLOGY**	**25**
	6.1 Processor Workload Selection and Execution	25
	6.2 Event Collection and Counter Initialization	25
	6.3 Performance Metric Calculation	25
	6.4 Architectural and Microarchitectural Analysis	28
	6.5 Simulation-Based Verification	28
	6.6 Flowchart of the System	29

CHAPTER No.	TITLE	PAGE No.
7	**SYSTEM DESIGN AND IMPLEMENTATION**	**30**
	7.1 Functional Block Diagram	30
	7.2 Wrapper and Processor Integration	31
	7.3 Event Collector and Event Counter Implementation	31
	7.4 Analytics Engine and Mode Controller Implementation	31
	7.5 Register Bank and Complete RTL Integration	32
8	**RESULTS AND ANALYSIS**	**33**
	8.1 Simulation Setup	33
	8.2 Performance Metrics and Evaluation Criteria	33
	8.3 Simulation Waveforms and Register Outputs	33
	8.4 Comparative Results and Discussion	37
9	**APPLICATIONS**	**40**
	9.1 RISC-V Processor Design	40
	9.2 RTL Verification and Debugging	40
	9.3 Pipeline Performance Analysis	40
	9.4 Comparison of Instruction Workloads	40
	9.5 Computer Architecture Education	40
	9.6 Reusable Processor Monitoring	41
10	**ADVANTAGES AND LIMITATIONS**	**42**
	10.1 Advantages	42
	10.2 Limitations	43
	10.3 Summary	43
11	**CONCLUSION AND FUTURE SCOPE**	**44**
	11.1 Conclusion	44
	11.2 Future Scope	44
12	**REFERENCES**	**46**
13	**APPENDICES**	**47**
	13.1 Performance Metric Definitions and Calculation Examples	47
	13.2 Comparative Performance Results	48
	ANNEXURE - I	**49**

LIST OF FIGURES

Figure No.	List of Figures	Page No.
Figure 1	Overall Functional Block Diagram	24
Figure 2	Analytics IP Architecture	42
Figure 3	Event Collector Simulation Waveform	42
Figure 4	Event Counter Bank Waveform	42
Figure 5	Processor Microarchitectural Monitoring Output	43
Figure 6	Pipeline Architectural Monitoring Output	43
Figure 7	Mode Controller Waveform	44
Figure 8	Register Bank Simulation Output	44

LIST OF TABLES

Table No.	List of Tables	Page No.
Table 1.1	Comparison of Performance Evaluation Approaches	17
Table 4.1	Event Classification and Signal Mapping	28
Table 8.1	Comparison of Performance-Monitoring Results	45

LIST OF ABBREVIATIONS

Abbreviation	Expansion
CPI	Cycles Per Instruction
CSV	Comma-Separated Values
FE	Forwarding Efficiency
HDL	Hardware Description Language
ID/EX	Instruction Decode / Execute
IF	Instruction Fetch
IP	Intellectual Property
IPC	Instructions Per Cycle
ISA	Instruction Set Architecture
MEM	Memory Access
PMU	Performance Monitoring Unit
PPMU	Programmable Performance Monitoring Unit
RAW	Read After Write
RISC-V	Reduced Instruction Set Computer – Five
RTL	Register Transfer Level
RV32I	RISC-V 32-bit Base Integer Instruction Set
WB	Write Back

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND AND CONTEXT OF RISC-V PROCESSORS

RISC-V is an open instruction set architecture (ISA) that defines the instructions a processor can execute and the behavior associated with those instructions. It provides a flexible foundation for developing processors for embedded systems, computer architecture research, educational applications, and high-performance computing. Unlike a fixed commercial processor implementation, RISC-V allows designers to develop customized processor cores according to their application requirements.

The RV32I instruction set is the 32-bit base integer instruction set of RISC-V. It supports fundamental operations such as arithmetic, logical operations, data transfer, conditional branching, and jump instructions. These instructions are processed by the processor's datapath and control logic to perform the required computation.

A pipelined processor divides instruction execution into multiple stages. Each stage performs a specific part of instruction processing, allowing different instructions to occupy different stages during the same clock cycle. This overlapping execution increases instruction throughput compared with a non-pipelined implementation.

However, pipelining introduces several challenges. Data dependencies may prevent an instruction from proceeding immediately, branch instructions may require changes in the program counter, and pipeline stalls or flushes may interrupt normal instruction flow. Forwarding mechanisms help resolve certain data dependencies by transferring results directly between pipeline stages.

Evaluating these effects is essential for understanding the actual performance of a processor. The total number of executed instructions alone does not indicate whether the pipeline is operating efficiently. Two processors executing the same instruction workload may require different numbers of clock cycles because of differences in their pipeline organization, hazard handling, and forwarding mechanisms.

Performance monitoring provides a systematic method of measuring these characteristics. By observing processor events and calculating derived metrics, designers can identify performance bottlenecks and make informed decisions about processor optimization. The proposed project develops a configurable Performance Monitoring and Analytics IP that collects processor execution events and converts them into meaningful performance indicators.

The analytics IP operates as an independent monitoring block connected to the processor through a wrapper interface.

1.2 PROJECT MOTIVATION AND PROBLEM STATEMENT

The motivation for this project is the need for a simple, reusable, and automated approach to evaluating the performance of pipelined RISC-V processors.

During processor verification, engineers commonly examine simulation waveforms to determine when instructions are executed, when hazards occur, and how the pipeline responds to stalls and flushes. Although waveform analysis is useful for debugging, manually inspecting numerous signals over thousands of clock cycles is time-consuming.

Conventional performance counters can record the number of clock cycles, retired instructions, branches, loads, stores, and other events. However, raw counter values do not always provide a complete explanation of processor behavior. Additional calculations may be required to determine IPC, CPI, forwarding efficiency, hazard distribution, and pipeline-stage utilization.

The problem becomes more significant when the processor executes large workloads. Manual waveform inspection and external post-processing may need to be repeated for different programs and processor configurations. This increases analysis time and makes consistent comparison more difficult.

The project addresses the following problem:

To design and simulate a configurable RTL-based performance analytics IP that automatically collects processor events, counts architectural and microarchitectural activities, calculates performance metrics, and stores the results without modifying the functional RTL of the host RISC-V processor.

The proposed solution introduces a modular analytics architecture with a wrapper interface, an event collection mechanism, dedicated counters, a metric calculation engine, a mode controller, and a result register bank.

The system aims to simplify processor performance evaluation by presenting both raw event counts and derived metrics in an organized form.

1.3 PROJECT OBJECTIVES AND SUCCESS CRITERIA

The primary objective of this project is to develop a reusable performance monitoring and analytics IP for evaluating a pipelined 32-bit RV32I RISC-V processor through RTL simulation. The project aims to study the processor architecture and its execution behavior, identify the architectural and microarchitectural signals required for performance monitoring, and develop a wrapper interface that connects the processor signals to the analytics IP without altering the processor's functional behavior. An Event Collector will be designed to identify relevant execution events, while an Event Counter Bank will record important parameters such as clock cycles, instructions, memory operations, branches, hazards, stalls, pipeline flushes, and forwarding activity. An Analytics Engine will be implemented to calculate performance metrics, including Instructions Per Cycle (IPC), Cycles Per Instruction (CPI), pipeline efficiency, stage utilization, hazard distribution, memory operation ratio, and forwarding efficiency. A Mode Controller will enable the selection of Architectural Profiling or Microarchitectural Analysis, and a Register Bank will store the raw counter values and calculated performance metrics for further evaluation. The functionality of individual modules and the integrated system will be verified through RTL simulation. Furthermore, processor performance results will be

compared across different instruction workloads to identify pipeline inefficiencies and provide useful insights for processor optimization. The completed design will be evaluated based on functional correctness, metric accuracy, mode selection, reset behavior, integration correctness, and workload consistency, ensuring that the monitoring IP produces reliable, reproducible results without affecting the functional execution of the host processor.

1.4 EXISTING APPROACHES AND COMPARATIVE ANALYSIS

Processor performance can be evaluated using several approaches, including manual waveform analysis, conventional hardware performance counters, software profiling, and integrated analytics systems.

1.4.1 Manual waveform analysis

In manual analysis, engineers inspect simulation waveforms to observe processor signals such as instruction retirement, stalls, flushes, forwarding activity, and pipeline state transitions.

This approach provides detailed information about individual clock cycles. However, it becomes difficult to use for long simulations because many events must be counted and interpreted manually.

1.4.2 Conventional performance counters

Conventional performance counters accumulate the number of occurrences of selected events. For example, a counter can record the number of executed instructions or the number of clock cycles. These values are useful for basic performance evaluation. However, additional processing may be required to convert raw counts into higher-level indicators.

1.4.3 Software-based profiling

Software profiling tools analyze execution statistics and help identify performance problems. Such tools can provide valuable information about program execution, but their usefulness depends on the processor, operating system, available counters, and profiling support.

1.4.4 Proposed analytics IP

The proposed design integrates event collection, event counting, metric calculation, mode selection, and result storage into a single modular RTL architecture.

Table 1.1: Comparison of performance evaluation approaches

Feature	Manual waveform analysis	Basic event counters	Proposed analytics IP
Raw event collection	Manual observation	Supported for configured events	Automatic event collection
Event counting	Manual or scripted	Automatic	Automatic
IPC and CPI calculation	External calculation	May require external processing	Integrated analytics
Hazard analysis	Detailed but time-consuming	Depends on available counters	Dedicated hazard counters
Forwarding analysis	Waveform inspection	Often requires additional processing	Forwarding metrics
Pipeline-stage utilization	Manual calculation	Not always available	Integrated calculation
Multiple analysis modes	Manual procedure	Implementation -dependent	Configurable modes
Result organization	Waveforms and notes	Counter values	Organized register bank
Reuse across processor cores	Requires manual adaptation	Depends on implementatio n	Wrapper-based adaptation

The table represents a qualitative comparison of the approaches. It does not imply that every existing PMU lacks advanced analytics; some implementations already provide programmable event selection and derived performance information.

1.5 SCOPE AND METHODOLOGY OVERVIEW

The project focuses on the RTL design, integration, and simulation-based evaluation of a configurable performance monitoring and analytics Intellectual Property (IP) for RISC-V processors. The primary scope is to develop a reusable digital monitoring system that observes the execution behavior of a 32-bit, three-stage pipelined RV32I RISC-V processor and evaluates its performance without modifying the processor's functional operation. The proposed analytics IP includes five functional modules: an Event Collector, an Event Counter Bank, an Analytics Engine, a Mode Controller, and a Register Bank. A wrapper interface will be developed to connect the internal processor signals to the analytics IP, enabling the monitoring system to identify relevant execution events, collect performance data, calculate performance metrics, and store the results for further

analysis. The monitored events include instruction retirement, clock cycles, branch operations, load and store instructions, Read-After-Write (RAW) hazards, load-use hazards, control hazards, pipeline stalls, pipeline flushes, and forwarding operations. Based on these events, the Analytics Engine will calculate important performance indicators such as Instructions Per Cycle (IPC), Cycles Per Instruction (CPI), pipeline efficiency, stage utilization, hazard distribution, memory operation ratio, and forwarding efficiency. The Mode Controller will provide two operating modes, namely Architectural Profiling and Microarchitectural Analysis, allowing the user to select the required level of performance monitoring. The Register Bank will store both raw event counts and calculated metrics so that the results can be accessed and evaluated systematically. The scope

also includes testing the monitoring system with different instruction workloads to understand how variations in instruction count, instruction type, and execution behavior affect processor performance. The project is limited to digital RTL design, processor integration, and functional verification using simulation tools. It does not involve physical chip fabrication, semiconductor device manufacturing, printed circuit board assembly, or the development of a physical hardware prototype.

The methodology begins with a detailed study of the architecture and execution behavior of the selected 32-bit, three-stage pipelined RV32I processor. During this stage, the processor's pipeline organization, instruction execution flow, control signals, and available internal signals will be examined to identify the information required for performance monitoring. Based on this study, the relevant architectural and microarchitectural events will be defined, along with the conditions under which each event should be detected and counted. Next, a wrapper interface will be designed to connect the processor signals to the analytics IP while ensuring that the monitoring logic does not interfere with instruction execution, data processing, or control flow. The five functional modules of the analytics IP will then be developed independently. The Event Collector will detect the selected execution events, the Event Counter Bank will maintain the corresponding event counts, the Analytics Engine will calculate the required performance metrics, the Mode Controller will select the appropriate monitoring mode, and the Register Bank will store the collected and calculated results. After individual module verification, the analytics IP will be integrated with the processor RTL through the wrapper interface. The integrated design will be simulated using representative instruction programs of different sizes and instruction compositions to evaluate various execution scenarios. Simulation waveforms and internal signal values will be examined to confirm correct event detection, counter updates, reset behavior, mode selection, and data storage. The calculated performance metrics will be independently verified against their mathematical definitions and reference calculations to ensure accuracy. Finally, the results obtained from different workloads will be compared to identify changes in processor performance, determine the effects of hazards, stalls, flushes, and forwarding operations, and highlight potential pipeline inefficiencies. The observations, verification results, limitations, and possible improvements will be documented to demonstrate the functionality, reliability, and reusability of the proposed performance monitoring and analytics IP.

CHAPTER 2
LITERATURE SURVEY

2.1 RISC-V PERFORMANCE MONITORING AND EXISTING APPROACHES

Performance monitoring is an important part of processor design because it provides quantitative information about instruction execution and microarchitectural behavior.

RISC-V provides a flexible processor architecture in which implementations may differ in pipeline depth, execution units, hazard handling, memory systems, and performance-monitoring capabilities. Therefore, a monitoring framework should account for differences in processor organization while maintaining a consistent interpretation of the collected events.

The RISC-V privileged architecture defines hardware performance-monitoring facilities, including counters used to measure cycles, instructions, and implementation-relevant events. Processor implementations may provide additional event counters for branches, cache operations, stalls, and other internal activities.

Open-source RISC-V processors can also provide implementation-specific monitoring capabilities. These capabilities help developers investigate processor behavior during software execution and hardware verification.

Another approach is to combine hardware counters with software profiling tools. This allows engineers to collect execution statistics and analyze the performance of applications. However, the depth of the analysis depends on the events exposed by the processor and the processing performed by the software tools.

Research on CPI-based performance analysis has also demonstrated the value of separating different sources of processor inefficiency. Rather than examining only total CPI, designers can investigate the contribution of memory activity, instruction dependencies, control flow, and pipeline interruptions.

These approaches establish the importance of collecting meaningful execution events. The proposed project builds on this concept by integrating raw event collection and derived metric calculation into a dedicated analytics IP.

2.2 PROCESSOR PERFORMANCE METRICS AND EVENT DATA MODELS

A performance-monitoring system requires clearly defined events and metrics.

2.2.1 Architectural events

Architectural events describe instruction execution and program-level activity. Examples include:

- Total clock cycles.
- Retired instructions.
- Branch instructions.

- Load instructions.
- Store instructions.

These events help describe the workload executed by the processor.

2.2.2 Microarchitectural events

Microarchitectural events describe internal processor behavior. Examples include:

- Read-after-write (RAW) hazards.
- Load-use hazards.
- Control hazards.
- Pipeline stalls.
- Pipeline flushes.
- Forwarding opportunities.
- Successful forwarding operations.
- Activity within individual pipeline stages.

These events help explain why a processor requires a particular number of cycles to execute a program.

2.2.3 Derived performance metrics

Raw event counts can be combined mathematically to obtain derived metrics.

Instructions Per Cycle (IPC) measures the average number of instructions retired per clock cycle.

IPC = Retired Instructions / Total Clock Cycles

Cycles Per Instruction (CPI) measures the average number of clock cycles required per retired instruction.

CPI = Total Clock Cycles / Retired Instructions

For nonzero instruction and cycle counts, IPC and CPI are reciprocals when calculated from the same counts.

Pipeline efficiency expresses the measured throughput relative to a selected ideal throughput. In the project's reported results, an IPC of approximately 0.92 corresponds to approximately 92% of the ideal one-instruction-per-cycle throughput.

Forwarding efficiency measures successful forwarding operations relative to detected forwarding opportunities.

Forwarding Efficiency = (Successful Forwarding / Forwarding Opportunities) × 100% **Stage utilization** expresses the active cycles of a pipeline stage relative to the total observation cycles.

Stage Utilization = (Stage Active Cycles / Total Observation Cycles) × 100% **Memory operation ratio** expresses the proportion of instructions that perform memory operations relative to the selected instruction population. The exact numerator and denominator must be defined consistently in the implementation.

Hazard distribution describes the relative contribution of different hazard categories. Its interpretation depends on the categories included in the total and on whether events can overlap.

2.3 RESEARCH GAPS AND MOTIVATION FOR THE PROPOSED SYSTEM

The reviewed approaches demonstrate that hardware counters, software profiling, and waveform analysis are useful for evaluating processor performance. However, a monitoring flow may still require multiple tools and separate calculations to transform raw events into a unified performance report.

The main design challenges addressed by this project are:

1. **Dependence on manual processing:** Raw event counts may require additional calculations and waveform inspection.
2. **Limited integration of metrics:** Basic counters do not necessarily provide a unified view of architectural and microarchitectural performance.
3. **Processor-specific signal differences:** Different processor implementations may expose different internal signals and pipeline organizations.
4. **Difficulty comparing workloads:** Results must be recorded consistently to compare processor behavior under different instruction programs.
5. **Result accessibility:** Counter values and derived metrics should be organized for straightforward retrieval.

The proposed analytics IP addresses these challenges through modular RTL design. The wrapper isolates processor-specific signal mapping, while the analytics modules perform event counting, metric calculation, mode control, and result storage.

The intended contribution is a reusable monitoring framework that reduces dependence on manual post-processing and provides a consistent basis for evaluating the behavior of pipelined RISC-V processors.

CHAPTER 3
SYSTEM OVERVIEW

3.1 OVERALL SYSTEM ARCHITECTURE

The proposed system consists of a RISC-V processor, an instruction memory, a wrapper interface, and a Performance Monitoring and Analytics IP. The processor executes the supplied instruction workload, while the wrapper collects selected signals required for performance analysis.

The analytics IP is organized into five functional modules: Event Collector, Event Counter Bank, Analytics Engine, Mode Controller, and Register Bank.

The Mode Controller manages the selection of the analysis mode and the activation of the corresponding monitoring functions. Configuration and control signals determine which categories of performance events are collected and analyzed.

The processor continues to execute its instructions according to its original functional design. The analytics IP observes relevant signals and calculates statistics without modifying the processor's intended instruction execution.

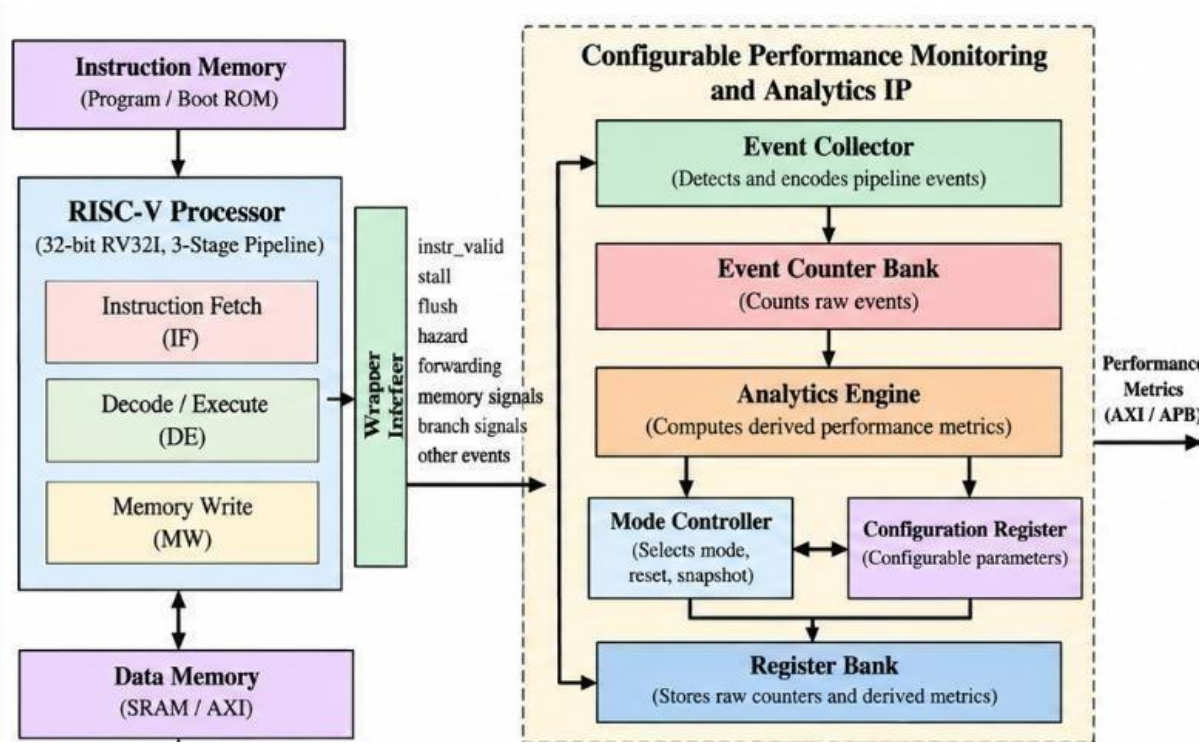


Figure 3.1: Overall Architecture of the Configurable Performance Monitoring and Analytics IP for RISC-V Processors.

Functional blocks

1. Instruction Memory: Stores the instruction program supplied to the processor. The processor retrieves instructions from memory and executes them according to its instruction-fetch mechanism.

2. RISC-V Processor: Executes RV32I instructions through its pipeline stages. The processor produces signals that indicate instruction activity, data dependencies, control-flow changes, stalls, flushes, and forwarding operations.

3. Wrapper Interface: Connects the processor's internal signals to the analytics IP. It maps processor-specific signals to the common event interface expected by the monitoring modules.

4. Event Collector: Receives the mapped signals and identifies relevant events during execution.

5. Event Counter Bank: Accumulates event occurrences over time.

6. Analytics Engine: Uses raw counter values to calculate derived performance indicators.

7. Mode Controller: Selects Architectural Profiling or Microarchitectural Analysis and manages the corresponding control signals.

8. Register Bank: Stores raw event counts and derived performance results for retrieval and examination.

3.2 DESIGN TECHNOLOGY AND TOOL SELECTION

The project is developed as a digital RTL design. It does not require physical hardware assembly to evaluate the logical operation of the analytics modules.

Hardware description language: Verilog or SystemVerilog can be used to describe the counters, combinational calculations, sequential control logic, and register storage.

RTL simulation: A simulator executes the HDL design according to its clock, reset, input signals, and testbench stimuli. The resulting waveforms are used to verify event detection and control behavior.

Testbench: A testbench supplies the clock, reset, control inputs, and processor activity required to evaluate the design. It also checks whether counter values and derived metrics match expected results.

Waveform viewer: The waveform viewer helps identify the timing relationships among processor events, counter updates, mode-selection signals, and register outputs.

RTL analysis and debugging: Compiler and simulator messages are used to identify syntax errors, incorrect signal connections, width mismatches, and simulation-time functional problems.

The specific tool and version used for each experiment should be recorded in the final report according to the actual simulation environment.

3.3 PERFORMANCE REGISTER ORGANIZATION AND DATA MANAGEMENT

The Register Bank is responsible for providing an organized and structured representation of the performance monitoring results generated by the analytics IP. It stores both raw event counter values and derived performance metrics, enabling the user to examine the execution behavior and efficiency of the 32-bit, three-stage pipelined RV32I RISC-V processor. The raw counter registers record important execution parameters, including total clock cycles, retired instructions, branch count, load count, store count, Read-After-Write (RAW) hazard count, load-use hazard count, control hazard count, stall count, flush count, forwarding opportunities, successful forwarding operations, and the active cycles of the monitored pipeline stages. These values provide detailed information about instruction execution, pipeline activity, data dependencies, control-flow changes, and forwarding behavior. In addition to storing raw counter values, the Register Bank maintains derived performance metrics calculated by the Analytics Engine, such as Instructions Per Cycle (IPC), Cycles Per Instruction (CPI), pipeline efficiency, instruction-fetch-stage utilization, decode/execute-stage utilization, memory-stage utilization, and write-back-stage utilization wherever applicable to the processor architecture. It also stores forwarding efficiency, hazard distribution, and memory operation ratio to support a more detailed analysis of processor performance. The register organization must clearly define the width, purpose, reset value, and update conditions of each register to ensure accurate and consistent data storage. The RTL implementation must also specify when counter values are updated, when calculated metrics are captured, and under what conditions the stored results become available for examination. A consistent register map, together with clearly defined read and update mechanisms, simplifies functional verification, improves accessibility of performance data, and enables systematic comparison of processor behavior across different instruction workloads. This organized storage mechanism makes the Register Bank an essential component of the proposed performance monitoring and analytics IP.

CHAPTER 4

PROCESSOR EVENT SOURCES AND INTEGRATION ARCHITECTURE

4.1 PROCESSOR PIPELINE SIGNAL INTEGRATION

The analytics IP obtains its input information from the signals generated by the RISC-V processor. These signals represent instruction execution and the internal conditions that affect pipeline progress.

The processor used in the project is a 32-bit, three-stage pipelined RV32I design. The exact names of the pipeline registers and signals depend on the host processor RTL. The wrapper acts as an intermediate interface between the processor and the analytics IP. It receives processor signals and maps them to the monitoring inputs.

A wrapper-based design is useful because it separates processor-specific signal names from the common analytics interface. When adapting the analytics IP to another processor, the wrapper can be modified to match the new core's signals while keeping the core analytics modules largely unchanged. The wrapper must preserve the original signal meanings and timing. A retired-instruction event, for example, must correspond to a valid completed instruction rather than merely an instruction present in a pipeline register.

4.2 EVENT CLASSIFICATION AND SIGNAL MAPPING

The monitored events are classified into architectural and microarchitectural categories.

Table 4.1: Event classification and signal mapping

Event category	Example event	Purpose
Execution	Clock cycles	Measures elapsed processor cycles
Instruction	Retired instructions	Measures completed instruction execution
Control flow	Branch events	Evaluates branch activity
Memory	Load and store events	Measures memory instruction activity
Data dependency	RAW hazards	Identifies read-after-write dependencies
Load dependency	Load-use hazards	Identifies dependencies involving load results
Control dependency	Control hazards	Measures control-flow-related pipeline events

Event category	Example event	Purpose
Pipeline progress	Stall events	Identifies cycles in which pipeline progress is restricted
Pipeline recovery	Flush events	Measures discarded or invalidated pipeline activity
Data transfer	Forwarding events	Evaluates the use of forwarding mechanisms
Stage activity	Active pipeline cycles	Measures pipeline-stage utilization

The event definitions must be documented carefully. For example, the number of hazard detections, the number of hazard cycles, and the number of affected instructions are not necessarily equal.
Similarly, forwarding opportunities should be counted according to the forwarding conditions implemented in the processor. A successful forwarding event should represent a qualifying forwarding operation, not simply an asserted forwarding-control signal unrelated to a valid instruction.

4.3 EVENT VALIDATION AND COUNTER UPDATE LOGIC

Each event counter requires a defined update condition.
A typical synchronous counter follows this conceptual behavior:

- On reset, the counter is initialized to its defined reset value.
- When the monitoring interval is active, the counter evaluates its associated event.
- If the event condition is satisfied, the counter increments.
- If the event condition is not satisfied, the counter retains its previous value.
- At the end of the monitoring interval, the accumulated value becomes available for analysis.

The exact implementation depends on whether the design supports pausing, clearing, or snapshotting counters during execution. For a cycle-based event, the counter may increment whenever the corresponding signal is asserted at a clock edge. For an instruction-based event, the counter should increment only when the instruction satisfies the defined qualification condition. The event logic must avoid counting the same logical event more than once unless the metric intentionally measures multiple cycles or multiple occurrences.

4.4 HANDLING RESET, STALLS, FLUSHES, AND EXECUTION COMPLETION

Reset establishes the initial state of the monitoring system. Counter values, mode-control state, and valid-result indicators should be initialized consistently with the design specification. Stalls require special attention because a stall can remain active for multiple clock cycles. A stall-cycle counter should count each qualifying stalled cycle. An event counter intended to count distinct stall episodes would require different logic. Flushes must also be interpreted correctly. A pipeline flush can invalidate instructions that were fetched or partially processed. These instructions should not be counted as retired instructions unless they subsequently complete according to the processor's actual retirement semantics. Execution completion identifies the end of the monitored instruction workload. It can be used to determine when the final counter values should be examined or when a result snapshot should be captured. If counters are cleared between workloads, each experiment starts from a defined baseline. If cumulative counters are retained, the report must identify which values belong to each monitoring interval. The correctness of the entire analytics IP depends on accurately interpreting these control conditions.

CHAPTER 5

SOFTWARE AND RTL DESIGN COMPONENTS

5.1 HARDWARE DESCRIPTION LANGUAGES

Hardware Description Languages (HDLs) are used to describe the structure and behavior of digital circuits. Verilog and SystemVerilog support the modeling of combinational logic, sequential logic, counters, registers, finite-state machines, and testbenches. In this project, HDL is used to describe the performance analytics IP and its connection to the RISC-V processor.

Verilog provides constructs such as module, input, output, reg, wire, always, and conditional statements to describe the circuit behavior. SystemVerilog extends Verilog with additional features, including stronger data types, enumerated types, structures, assertions, and more advanced verification constructs.

The RTL describes how the design operates on clock edges and how signals influence the counters and control logic. The simulator executes this description to evaluate the behavior of the design over time.

5.1.1 Combinational logic

Combinational logic produces outputs based on the current input values. In the analytics IP, it may be used for event qualification, metric calculation, and selection of intermediate values.

5.1.2 Sequential logic

Sequential logic stores state and updates it according to clock and control signals. Event counters, configuration registers, and control-state registers are examples of sequential logic.

5.1.3 Counter design

Counters record event occurrences. They are generally implemented as registers that increment when a qualifying event is detected.

A counter must have sufficient width for the maximum expected count. Counter overflow and reset behavior should be considered when selecting its width.

5.2 SIMULATION AND VERIFICATION ENVIRONMENT

RTL simulation verifies the logical behavior of a design before any physical implementation is considered. The simulation environment contains the design under test, a clock generator, reset logic, input stimulus, and checking logic. The testbench applies the required inputs and observes the outputs. It may compare the actual values with expected values and report whether each test passes or fails. The following test categories are appropriate for the analytics IP:

Reset verification: Confirms that counters and control registers enter their specified initial states.

Event-count verification: Applies a known sequence of event signals and checks the resulting counter values.

Mode-selection verification: Checks that Architectural Profiling and Microarchitectural Analysis are selected correctly.

Metric verification: Compares calculated IPC, CPI, forwarding efficiency, and stage utilization against reference calculations.

Integration verification: Runs the analytics IP with the processor and checks whether the collected events correspond to the processor's behavior.

Workload verification: Executes different instruction programs and compares the resulting performance statistics.

5.3 RTL DESIGN MODULES

The complete design is divided into functional modules to improve readability, verification, and reuse.

5.3.1 Event Collector module

This module receives the processor activity signals and produces qualified event indications. It must follow the event definitions established for the design.

5.3.2 Event Counter Bank module

This module stores the accumulated event counts. Separate counters allow the design to distinguish instruction execution, memory operations, hazards, stalls, flushes, forwarding operations, and stage activity.

5.3.3 Analytics Engine module

This module calculates the derived performance indicators using the raw counter values. It must handle zero denominators and use consistent arithmetic widths.

5.3.4 Mode Controller module

This module generates the control signals that select the monitoring mode and manage counter operation.

5.3.5 Register Bank module

This module stores the monitoring results and provides a defined way to inspect the raw and derived values.

5.3.6 Wrapper module

The wrapper maps the processor-specific signals to the generic monitoring interface. It is the main adaptation point when the analytics IP is connected to a different processor implementation.

5.4 DEVELOPMENT AND DEBUGGING TOOLS

The development environment should include an HDL editor, an RTL simulator, a waveform viewer, and tools for examining compilation and simulation messages.

The HDL editor is used to create and maintain the source modules. The simulator checks whether the RTL compiles and behaves according to the expected timing and logic. The waveform viewer provides a time-based representation of the internal signals.

During debugging, the designer examines the sequence of processor events, event-counter updates, mode-control signals, and stored metric values. When a discrepancy occurs, the corresponding input event and counter update condition can be inspected to determine the cause.

The final report should list the exact tools used for the project rather than including tools that were not used.

CHAPTER 6
METHODOLOGY

6.1 PROCESSOR WORKLOAD SELECTION AND EXECUTION

The processor workload is the instruction program used to evaluate the performance analytics IP. The project paper reports evaluation using a 32-bit, three-stage pipelined RV32I processor and two workload sizes: approximately 2K and 10K instructions. Different workload sizes are useful because they allow the monitoring framework to be evaluated over different amounts of instruction execution. A larger workload also provides more opportunities to observe branches, memory operations, hazards, stalls, and forwarding activity. Before each experiment, the processor and analytics IP must be placed into a defined initial state. The instruction program is then executed, and the analytics IP observes the required events.

The same event definitions should be used across experiments to make the results comparable.

6.2 EVENT COLLECTION AND COUNTER INITIALIZATION

The monitoring process begins by resetting the analytics IP and configuring the required analysis mode. After initialization, the processor begins executing the instruction workload. The wrapper passes the relevant signals to the Event Collector, which generates event indications according to the implemented qualification conditions. The Event Counter Bank accumulates the occurrences of the selected events. For example, when a qualifying instruction retires, the retired-instruction counter increments. When a qualifying load instruction is identified, the load counter increments. When the pipeline experiences a qualifying stall cycle, the stall counter records that cycle. The counters continue to accumulate values until the monitoring interval ends or the implemented control logic stops counting. At the end of the experiment, the accumulated values are made available to the Analytics Engine and Register Bank.

6.3 PERFORMANCE METRIC CALCULATION

The Analytics Engine calculates the derived metrics from the collected event counts.

6.3.1 Instructions Per Cycle

IPC measures the average number of retired instructions per clock cycle.

Here, $N_{retired}$ represents the number of retired instructions and N_{cycles} represents the total measured clock cycles.

A higher IPC generally indicates greater instruction throughput for the evaluated workload, provided the workloads and measurement conditions are comparable.

6.3.2 Cycles Per Instruction

CPI measures the average number of clock cycles required for each retired instruction. A lower CPI generally indicates better instruction throughput under comparable conditions. When both metrics use the same nonzero counts, CPI is the reciprocal of IPC.

6.3.3 Pipeline efficiency

For the project's reported metric, pipeline efficiency is calculated relative to the ideal throughput of one retired instruction per cycle.

If $IPC_{ideal} = 1$, then an IPC of 0.92 corresponds to 92% efficiency.

This is a throughput-based efficiency measure. It should not be confused with the percentage of cycles in which any pipeline stage is active.

6.3.4 Forwarding efficiency

Forwarding efficiency measures the proportion of detected forwarding opportunities that are successfully resolved through forwarding.

$$\text{Forwarding Efficiency} = \frac{N_{\text{successful forwarding}}}{N_{\text{forwarding opportunities}}} \times 100\%$$

A value of 100% indicates that all opportunities included in the measured denominator were counted as successful. It does not establish that all possible data hazards in the processor were eliminated.

6.3.5 Pipeline-stage utilization

Stage utilization measures the activity of a pipeline stage during the observation

Here, $N_{active,i}$ is the active-cycle count for stage i.

The interpretation depends on how the design defines an active cycle. For a three-stage processor, the report must use the actual stage organization of the host core rather than assuming that a five-stage pipeline is present.

6.3.6 Hazard distribution

Hazard distribution describes the relative contribution of a hazard category to the selected total.

$$\text{Hazard Distribution}_i = \frac{N_{\text{hazard},i}}{N_{\text{hazard,total}}} \times 100\%$$

The numerator and denominator must use compatible counting definitions. If multiple hazard signals can be asserted simultaneously, the report must explain whether the categories are mutually exclusive or overlapping.

6.3.7 Memory operation ratio

The memory operation ratio indicates the proportion of monitored instructions that perform load or store operations.

One common definition is:

$$\text{Memory Operation Ratio} = \frac{N_{\text{load}} + N_{\text{store}}}{N_{\text{retired}}} \times 100\%$$

This definition is valid when loads, stores, and retired instructions are counted consistently. The exact formula used in the final implementation should be documented.

6.4 ARCHITECTURAL AND MICROARCHITECTURAL ANALYSIS

The proposed system supports two analysis modes.

Architectural Profiling

Architectural Profiling focuses on the overall execution characteristics of the instruction program. It includes metrics such as cycle count, retired instructions, branches, loads, stores, IPC, and CPI.

These metrics help compare execution throughput across workloads and processor configurations.

Microarchitectural Analysis

Microarchitectural Analysis examines the internal behavior of the pipeline. It includes hazard counts, stall cycles, flush events, forwarding activity, and stage utilization.

For example, a high RAW hazard count may indicate frequent data dependencies. A high control-hazard count may indicate substantial control-flow activity or associated pipeline recovery, depending on the event definition.

By examining these statistics together, the designer can identify which aspects of the processor deserve further investigation.

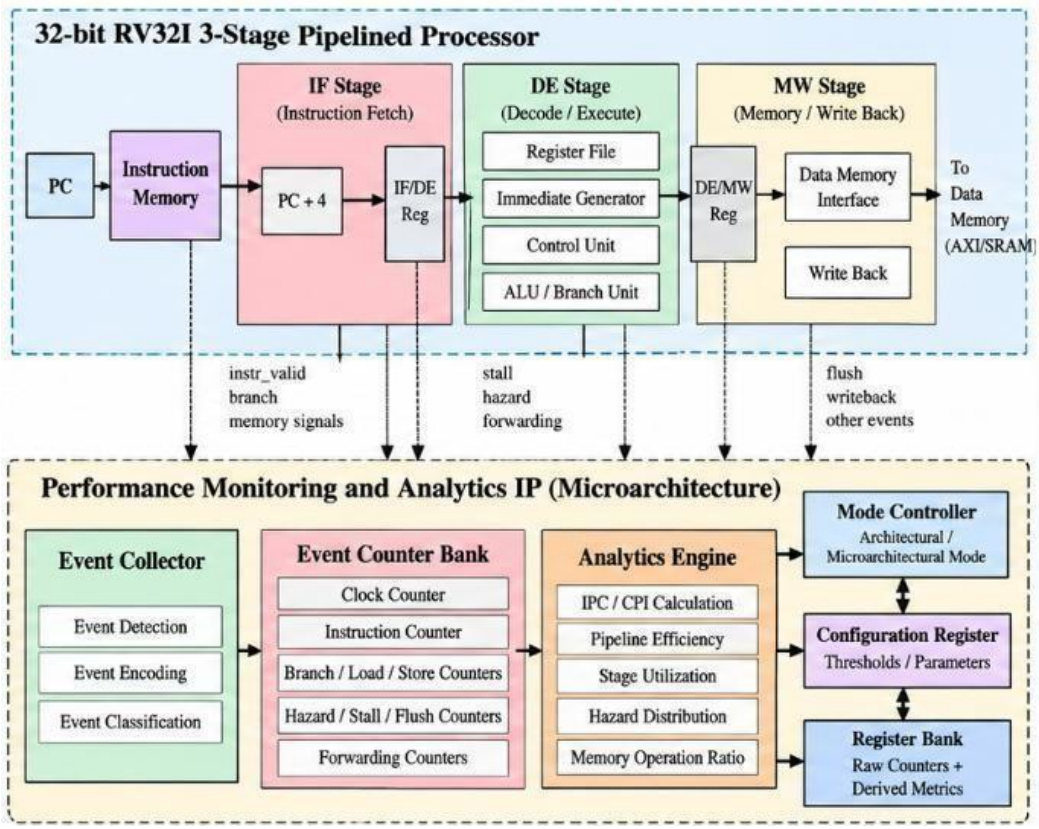


Figure 6.1 : Microarchitecturural analysis

6.5 SIMULATION-BASED VERIFICATION

Verification is performed at two levels.

Module-level verification: Each analytics module is tested separately to confirm its intended behavior. Counter logic, mode control, metric arithmetic, and register storage are checked using controlled inputs.

System-level verification: The complete analytics IP is connected to the processor through the wrapper. The processor executes the selected workload, and the monitoring outputs are compared against the observed processor events.

For every experiment, the final event counts should be checked against the signal activity and the corresponding update conditions. Derived metrics should then be recomputed independently from the raw counts.

This approach helps distinguish errors in event detection from errors in arithmetic or result storage.

6.6 FLOWCHART OF THE SYSTEM

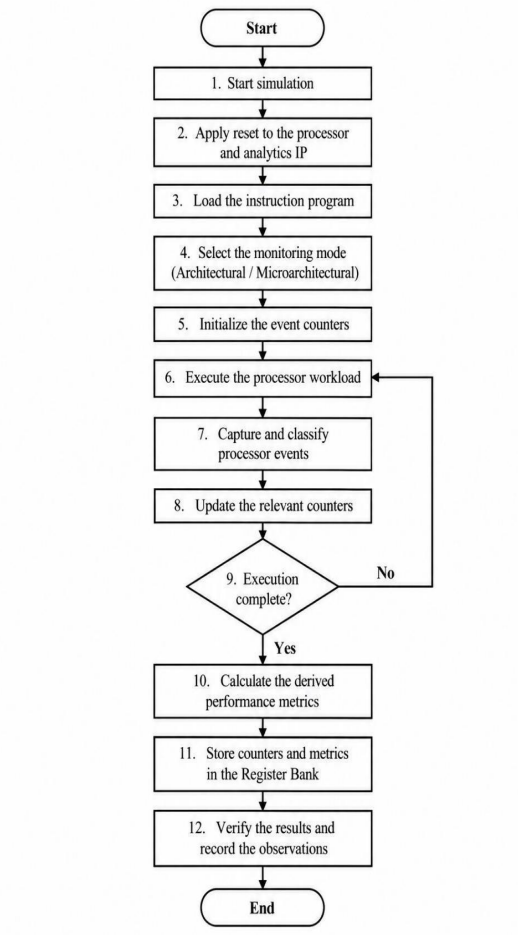


Figure 6.2 Flow chart

CHAPTER 7
SYSTEM DESIGN AND IMPLEMENTATION

7.1 FUNCTIONAL BLOCK DIAGRAM

The proposed architecture separates processor execution from performance analysis. The processor executes the instruction workload, while the analytics IP collects and evaluates selected signals.

The functional block diagram should contain the following blocks:

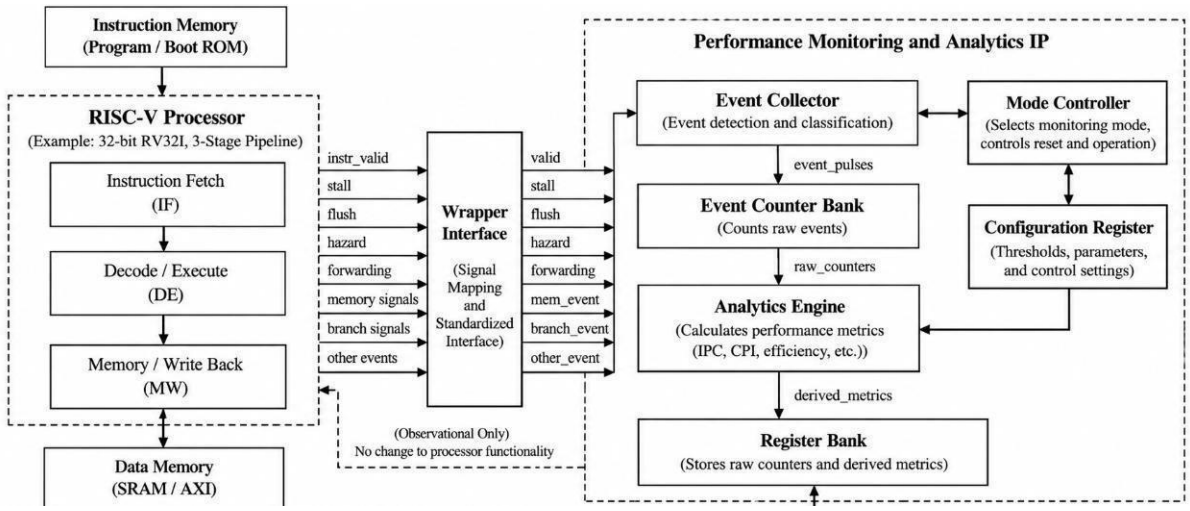


Figure 7.1: Functional block diagram of the analytics IP

The Mode Controller supplies control signals to the relevant analytics modules. The exact connections depend on the implemented RTL. The diagram is a functional representation rather than a pin-level schematic.

7.2 WRAPPER AND PROCESSOR INTEGRATION

The wrapper is responsible for connecting the processor's internal activity signals to the analytics IP.

Different processors may use different signal names and pipeline structures. Therefore, the wrapper provides a common interface between the processor and the Performance Monitoring and Analytics IP, allowing the analytics module to operate independently of the processor's internal naming conventions. The integration process begins by identifying the processor signals required for monitoring and determining their meaning and timing. These signals are then mapped to the wrapper interface, whose outputs are connected to the Event Collector for detecting and classifying processor events. The detected events are passed to the Event Counter Bank, which maintains the raw event counts. These counters are supplied to the Analytics Engine to calculate performance metrics. The Mode Controller and Configuration Register manage the monitoring mode and configuration parameters, while the calculated results are stored in the Register Bank for retrieval and analysis. Finally, the complete integrated design is simulated, and the output values are inspected to verify correct operation. The wrapper must remain observational and non-intrusive, meaning it must not modify the processor's functional inputs or interfere with instruction execution. This approach allows the same analytics architecture to be reused with other compatible RISC-V processors by modifying only the signal-mapping layer.

7.3 EVENT COLLECTOR AND EVENT COUNTER IMPLEMENTATION

The Event Collector receives the wrapper signals and identifies valid performance events. The collected event indications are passed to the counter bank. Each counter is associated with a specific event definition. The implementation should distinguish between events counted once per instruction and events counted once per cycle. For example, an instruction counter should not increment repeatedly simply because an instruction remains in a pipeline stage across multiple cycles. Conversely, a stall-cycle counter may need to increment on every cycle that the stall condition is active. The Event Counter Bank uses sequential logic to preserve the accumulated counts. Reset and counter-enable conditions determine when counting begins and ends. The counters should also be designed to prevent unintended counting during reset, invalid pipeline states, or periods when monitoring is disabled.
After the workload completes, the final values represent the accumulated event counts for that monitoring interval.

7.4 ANALYTICS ENGINE AND MODE CONTROLLER IMPLEMENTATION

The Analytics Engine receives the event-counter values and calculates the selected performance indicators. Its implementation may use combinational arithmetic to calculate metrics or

sequential logic to capture results at a defined point in time. The design must ensure that the raw counter values used for a calculation belong to the same measurement interval. The engine must also handle arithmetic corner cases. If the retired-instruction count is zero, PI is undefined. If the forwarding-opportunity count is zero, forwarding efficiency cannot be calculated using the normal ratio. The implementation should use a defined invalid-result indicator or another documented convention rather than silently treating an undefined value as a meaningful result. The Mode Controller determines which analysis mode is active and coordinates the associated enables. Architectural Profiling focuses on instruction execution and aggregate throughput. Microarchitectural Analysis focuses on pipeline events and resource utilization. The controller should ensure that reset, mode selection, counter activation, and result capture follow a deterministic sequence. If mode selection is permitted during execution, the design must define how counters behave during a mode change so that the resulting statistics remain interpretable.

7.5 REGISTER BANK AND COMPLETE RTL INTEGRATION

The Register Bank stores the raw counters and derived metrics produced by the analytics IP. A complete implementation requires consistent signal widths, correct module-port connections, and clearly defined reset behavior. The result registers must capture valid data and retain it until the next specified update or reset.

The RTL integration process follows this order:

1. Compile the individual modules.
2. Verify the port declarations and signal widths.
3. Connect the analytics modules in the intended data flow.
4. Integrate the wrapper with the host processor.
5. Create a testbench for the complete design.
6. Apply reset and start the selected workload.
7. Observe event-counter updates.
8. Verify mode-controller behavior.
9. Check the calculated performance metrics.
10. Inspect the final register values.

The implementation is considered functionally validated only when the simulation results agree with the expected event counts and independently checked calculations.

No physical hardware implementation is required for this project scope. The evaluation is based on RTL simulation and the interpretation of the resulting digital signals.

CHAPTER 8
RESULTS AND ANALYSIS

8.1 SIMULATION SETUP

The project paper reports simulation-based evaluation of the analytics IP using a 32-bit, three-stage pipelined RISC-V processor implementing the RV32I instruction set. The reported evaluation includes both architectural and microarchitectural metrics. The raw values are used to assess instruction execution and pipeline behavior, while the derived values provide a more concise representation of performance. The final report should include the actual simulation tool, version, clock settings, reset conditions, workload files, and testbench configuration used in the experiments.

8.2 PERFORMANCE METRICS AND EVALUATION CRITERIA

The principal performance metrics are IPC, CPI, pipeline efficiency, stage utilization, forwarding efficiency, hazard distribution, and memory operation ratio. Forwarding efficiency evaluates the forwarding opportunities detected by the monitoring logic. Hazard distribution helps identify the relative contribution of data and control dependencies. The raw counters are important because derived metrics alone may not explain why performance changes. For example, a reduced IPC may be associated with increased stalls, control-flow activity, or other execution effects. The counter values help the designer investigate these possibilities.

8.3 SIMULATION WAVEFORMS AND REGISTER OUTPUTS

The project paper includes simulation outputs for the Event Collector, Event Counter Bank, processor monitoring, pipeline architectural monitoring, Mode Controller, and Register Bank. The following figures can be included in the final project report if the corresponding simulation screenshots are available.

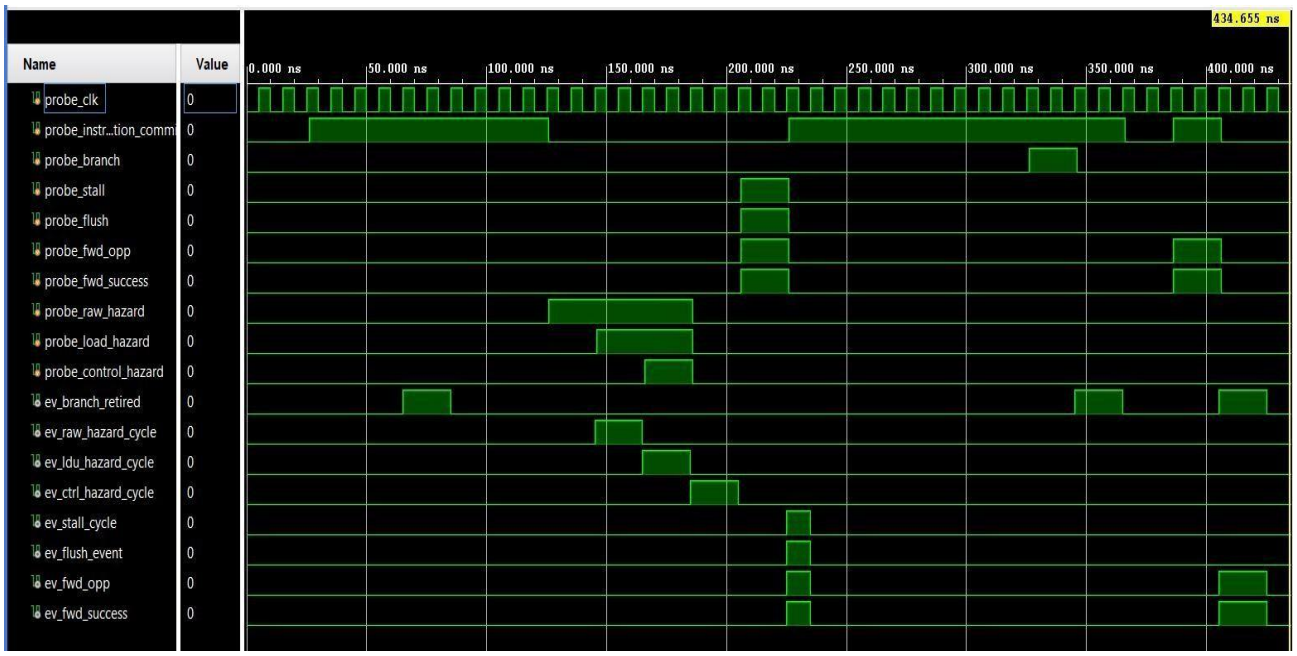


Figure 8.1: Event Collector simulation waveform. Demonstrates how processor-level signals are captured and converted into event indications.

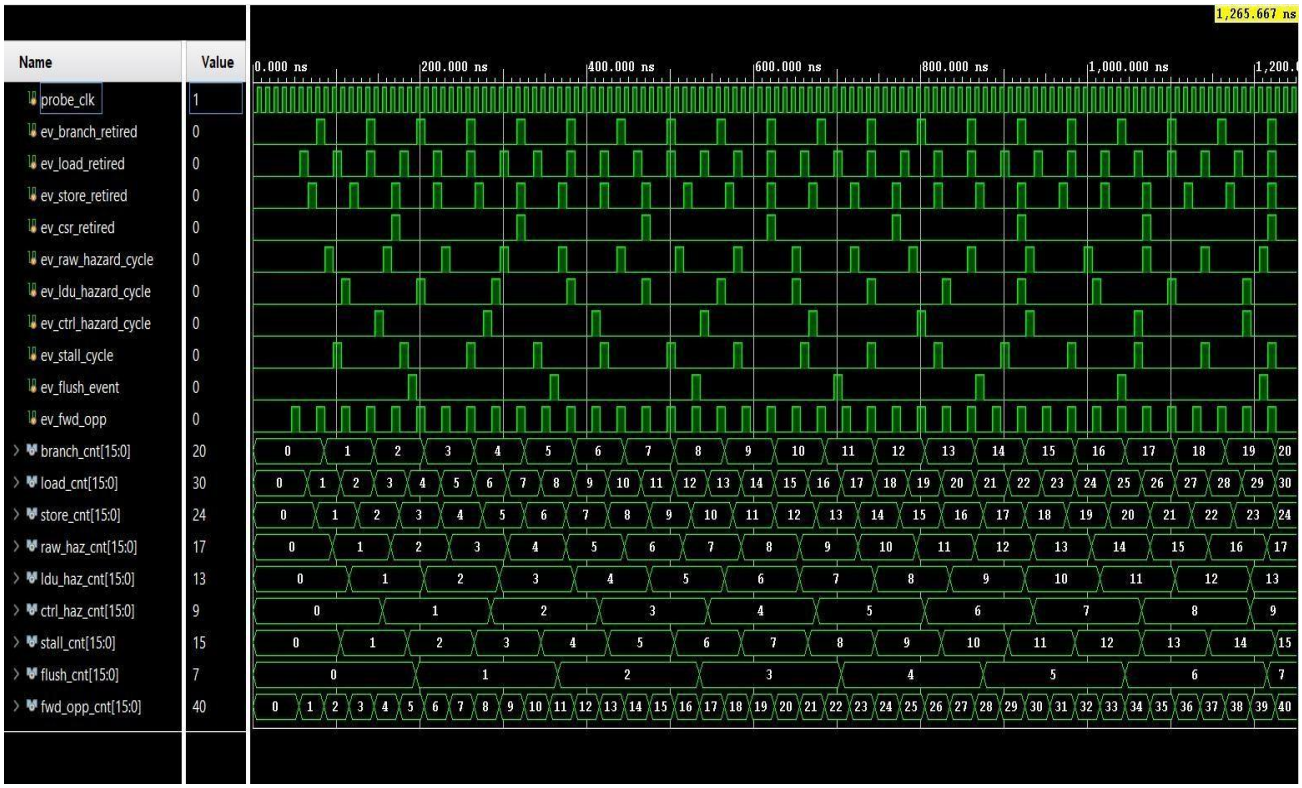


Figure 8.2: Event Counter Bank waveform. Shows the accumulation of events such as hazards, stalls, branches, memory operations, and forwarding activity.

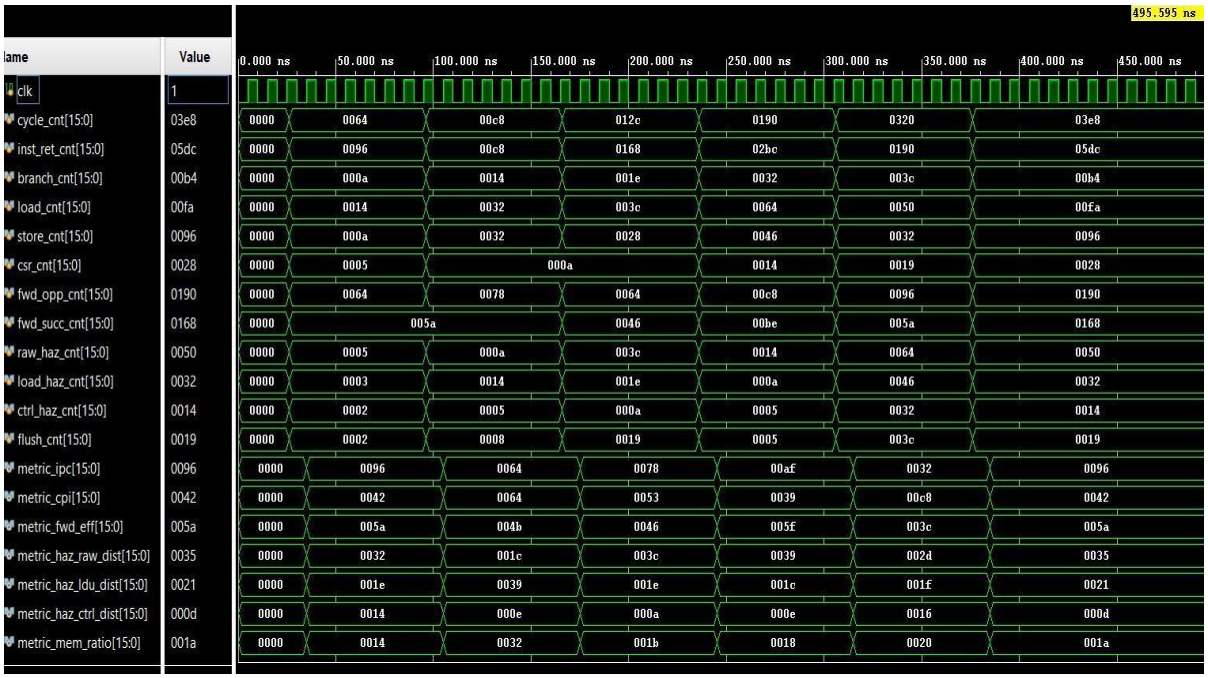


Figure 8.3: Processor microarchitectural monitoring output. Shows the collected counters and their evolution during processor execution.

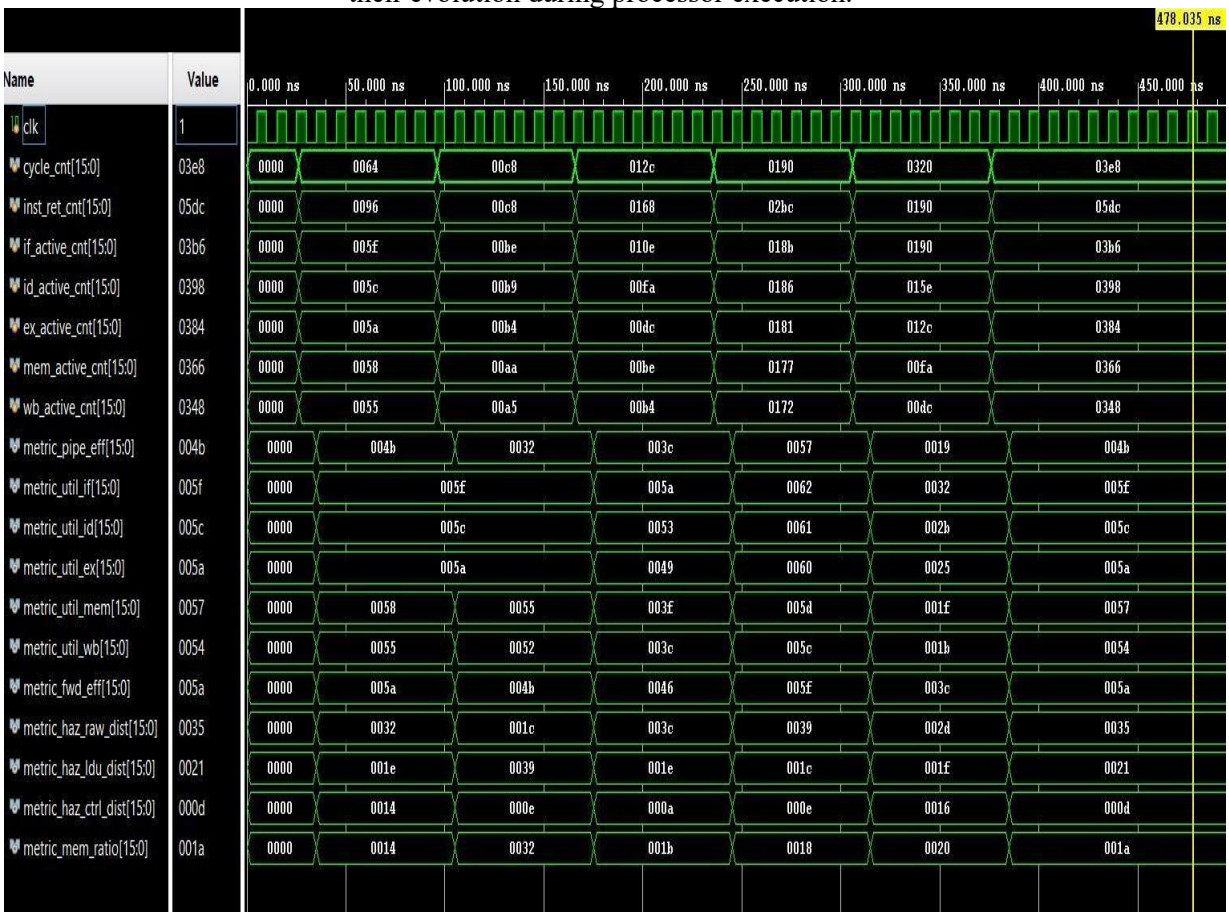


Figure 8.4: Pipeline architectural monitoring output. Illustrates the pipeline activity used to evaluate architectural and stage-level performance.

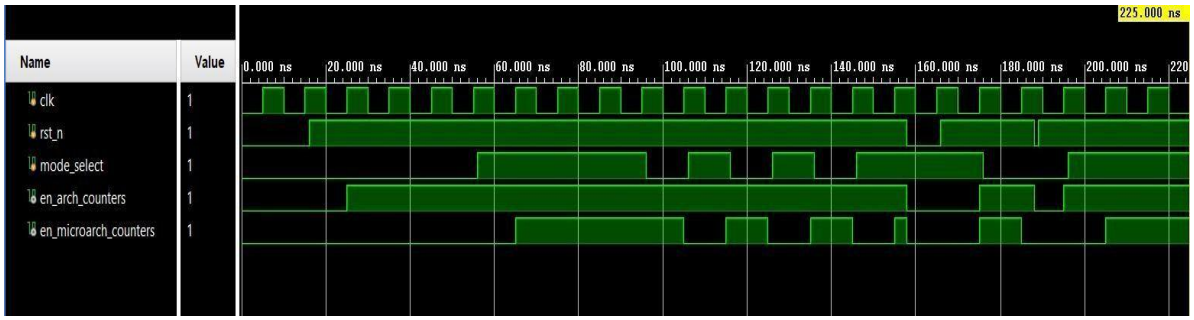


Figure 8.5: Mode Controller waveform. Shows the clock, reset, mode-selection signal, and monitoring-control signals.

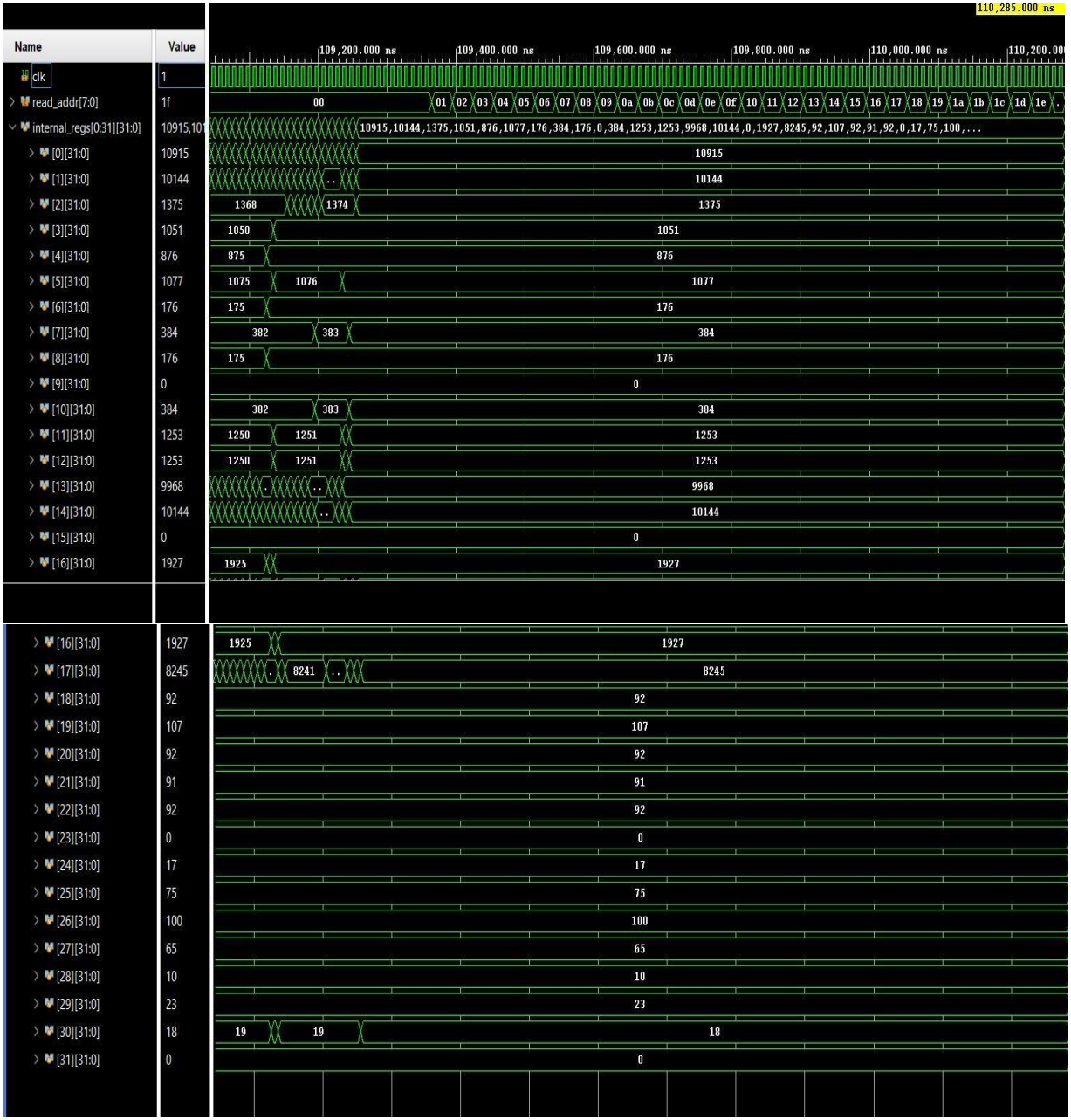


Figure 8.6: Register Bank simulation output. Demonstrates the storage of accumulated counters and calculated metrics

8.4 COMPARATIVE RESULTS AND DISCUSSION

The following table reproduces the principal values reported in the supplied project paper.

Table 8.1: Comparison of performance-monitoring results

Metric	10K-instruction workload	2K-instruction workload
Cycle count	10,915	2,198
Retired instructions	10,144	2,031
Branch count	1,375	216
Load count	1,051	331
Store count	876	112
RAW hazard count	1,077	475
Load-use hazard count	176	112
Control hazard count	384	81
Stall count	176	112
Flush count	384	81
Forwarding opportunities	1,253	502
Successful forwarding	1,253	502
IF active cycles	9,968	1,921
ID/EX active cycles	10,144	2,032
MEM active cycles	1,927	443
WB active cycles	8,245	1,757
IPC	0.92	0.92
CPI	1.07	1.08
Pipeline efficiency	92%	92%
IF utilization	91%	87%
ID/EX utilization	92%	92%
MEM utilization	17%	20%
WB utilization	75%	79%

Forwarding efficiency	100%	100%
RAW hazard distribution	65%	71%
Load-use hazard distribution	10%	16%
Control-hazard distribution	23%	12%
Memory operation ratio	18%	21%

Note: These are the values reported in the supplied paper, not newly generated experimental results. Some percentages are rounded, and the exact denominator and counting conventions should be confirmed against the RTL before independently reproducing every metric.

8.4.1 Analysis of instruction execution

The reported cycle count increases from 2,198 cycles for the 2K workload to 10,915 cycles for the 10K workload. The retired-instruction count also increases, from 2,031 to 10,144. This is consistent with the larger workload requiring more processor execution time. The relationship is not expected to be perfectly proportional because the actual instruction count, instruction mix, pipeline activity, and control-flow behavior affect the total number of cycles. The reported IPC is 0.92 for both workloads, while CPI is 1.07 and 1.08, respectively. These values indicate similar average throughput across the two evaluated cases.

8.4.2 Analysis of pipeline efficiency

The project reports pipeline efficiency of 92% for both workloads.
This result suggests that the measured instruction throughput is approximately 92% of the selected ideal throughput of one instruction per cycle. The remaining difference is associated with the processor's actual execution behavior, including pipeline dependencies and control-flow effects.
The value should be interpreted using the project's defined throughput-based efficiency formula.

8.4.3 Analysis of pipeline-stage utilization

The IF utilization is reported as 91% for the 10K workload and 87% for the 2K workload. The ID/EX utilization is reported as 92% for both workloads. The MEM utilization is lower, at 17% and 20%, respectively. This is consistent with memory instructions representing only part of the executed workload, although stage activity is also influenced by the processor's implementation and the precise utilization definition. The WB utilization is reported as 75% and 79%. These values reflect the activity definition and the pipeline organization used in the experiment. Stage utilization should not automatically be interpreted as a direct measure of stage efficiency. A stage may be idle because of the instruction mix, pipeline bubbles, control behavior, or other design-specific conditions.

8.4.4 Analysis of forwarding efficiency

The paper reports 1,253 forwarding opportunities and 1,253 successful forwarding operations for the 10K workload. For the 2K workload, it reports 502 opportunities and 502 successful operations. Under the project's counting convention, these values correspond to 100% forwarding efficiency. This indicates that all detected forwarding opportunities were counted as successful. It does not mean that all data dependencies were resolved without stalls, because some dependencies, including certain load-use cases, may require additional cycles.

8.4.5 Analysis of hazard distribution

RAW hazards represent the largest reported hazard category, accounting for 65% in the 10K workload and 71% in the 2K workload. The reported load-use hazard distribution is 10% and 16%, while the control-hazard distribution is 23% and 12%. These values indicate that data dependencies and control-flow behavior form important areas for further investigation. However, the distributions should be interpreted according to the actual counting method, especially if hazard categories overlap. The results can help guide investigation into instruction scheduling, forwarding logic, branch handling, and pipeline control.

8.4.6 Overall discussion

The reported results demonstrate the usefulness of collecting both raw processor events and derived performance metrics in a single analytics framework. The event counters quantify execution activity, while the Analytics Engine provides a concise interpretation of throughput, pipeline activity, and hazard behavior. The Register Bank organizes the results for comparison and verification. The wrapper-based architecture also provides a potential route to reuse the analytics IP with other RISC-V implementations, provided that their event signals can be mapped correctly and their pipeline semantics are understood. A final validation should independently recalculate the metrics from the raw counters and confirm that the definitions and denominators are consistent. This is particularly important when evaluating hazard distributions, stage utilization, and the relationship between IPC and CPI.

CHAPTER 9
APPLICATIONS

The Configurable Performance Monitoring and Analytics IP can be applied in several areas of processor design, verification, and optimization.

9.1 RISC-V PROCESSOR DESIGN

The analytics IP can help processor designers evaluate the behavior of a RISC-V core during instruction execution. By collecting cycle counts, retired instructions, hazards, stalls, and forwarding activity, the designer can identify characteristics that affect instruction throughput. For example, if the measured CPI is higher than expected, the designer can examine the hazard and stall counters to determine which events may contribute to the increased cycle count.

9.2 RTL VERIFICATION AND DEBUGGING

During RTL verification, the analytics IP can provide numerical evidence about processor behavior. Instead of relying exclusively on waveform inspection, a verification engineer can examine the number of observed stalls, flushes, branches, and forwarding events. These counts can be compared against the expected behavior of the test program. This approach can complement existing functional testbenches and improve the organization of simulation results.

9.3 PIPELINE PERFORMANCE ANALYSIS

The system can be used to evaluate pipeline throughput and stage activity. IPC and CPI describe overall throughput, while stage-utilization metrics provide additional information about pipeline activity. Hazard and stall statistics can help the designer investigate periods of reduced progress.The results can support decisions about pipeline control, forwarding mechanisms, instruction dependencies, and branch handling.

9.4 COMPARISON OF INSTRUCTION WORKLOADS

The analytics IP can evaluate multiple instruction workloads using consistent event definitions. For example, a workload containing many arithmetic instructions may exhibit different memory-operation and hazard distributions from a workload containing frequent loads, stores, and branches.
Comparing the resulting metrics helps explain how instruction mix influences processor behavior.

9.5 COMPUTER ARCHITECTURE EDUCATION

The project can serve as an educational framework for understanding pipelined processor operation. Students can relate instruction execution to cycle counts, forwarding activity, hazards, stalls, and flushes. The derived metrics provide a numerical connection between processor architecture concepts and observed simulation behavior.

9.6 REUSABLE PROCESSOR MONITORING

The wrapper-based design can be adapted to other compatible RISC-V implementations. This requires identifying the equivalent signals, mapping them to the analytics interface, and validating that the event definitions remain correct. Such reuse can reduce the effort required to develop a separate performance-analysis system for every processor implementation.

CHAPTER 10
ADVANTAGES AND LIMITATIONS

10.1 ADVANTAGES

10.1.1 Automated event collection

The Event Collector records selected processor events automatically during simulation. This reduces the need to count events manually from waveforms.

10.1.2 Integrated metric calculation

The Analytics Engine calculates derived performance indicators from the raw event counters. This simplifies the evaluation of IPC, CPI, pipeline efficiency, stage utilization, and forwarding efficiency.

10.1.3 Modular architecture

The separation of event collection, counter storage, metric calculation, mode control, and result storage makes the design easier to maintain and extend.

10.1.4 Reduced dependence on manual post-processing

The system provides an integrated method for calculating and organizing performance statistics. This reduces the need for separate scripts for every experiment, although independent scripts can still be useful for verification.

10.1.5 Architectural and microarchitectural analysis

The two operating modes allow the system to examine both overall instruction execution and internal pipeline behavior.

10.1.6 Reusability

The wrapper separates processor-specific signals from the analytics logic. This supports adaptation to different processor implementations when their signals can be mapped to the common interface.

10.1.7 Simulation-based verification

The design can be evaluated using RTL simulation without requiring physical hardware. This allows functional behavior and metric calculations to be examined in a controlled environment.

10.1.8 Support for optimization

The reported metrics can help identify potential performance bottlenecks and guide further investigation into hazards, stalls, forwarding, and pipeline-stage activity.

10.2 LIMITATIONS

10.2.1 Dependence on processor signals

The accuracy of the analytics IP depends on the availability and correctness of the signals supplied by the wrapper. Missing or incorrectly mapped signals can lead to inaccurate statistics.

10.2.2 Dependence on event definitions

Different processors may define instruction retirement, stalls, flushes, and forwarding events differently. Reuse therefore requires careful validation of the event semantics.

10.2.3 Finite counter width

Hardware counters have finite widths. If the maximum count exceeds the representable range, overflow can occur unless it is handled explicitly.

10.2.4 Arithmetic limitations

RTL calculations using integer arithmetic may introduce truncation. Limited register widths can also cause overflow during intermediate calculations.
The implementation must select appropriate widths and document the precision of the reported metrics.

10.2.5 Metric interpretation

A single metric does not necessarily explain the complete behavior of a processor. For example, high forwarding efficiency does not guarantee that the processor has no load-use stalls.
Derived values should be interpreted together with their underlying event counts.

10.2.6 Simulation-only evaluation

The present project scope evaluates the design through RTL simulation. It does not establish physical silicon performance, post-layout timing, power consumption, or actual silicon area.

10.2.7 Dependence on the workload

Performance results vary according to the instruction mix, data dependencies, branch behavior, and memory operations of the program. Results from one workload cannot automatically be generalized to all applications.

10.3 SUMMARY

The proposed analytics IP provides a structured approach to processor performance evaluation. Its main advantages are modularity, automatic event collection, integrated metric calculation, and wrapper-based adaptation. Its effectiveness depends on correct event definitions, appropriate counter widths, accurate arithmetic, and systematic simulation verification.

CHAPTER 11
CONCLUSION AND FUTURE SCOPE

11.1 CONCLUSION

The project presents the design and simulation-based evaluation of a Configurable Performance Monitoring and Analytics IP for RISC-V processors. The primary objective is to transform processor execution events into meaningful architectural and microarchitectural performance information without modifying the functional RTL of the host processor. The proposed architecture consists of five functional modules: Event Collector, Event Counter Bank, Analytics Engine, Mode Controller, and Register Bank. A wrapper interface connects the analytics IP to the processor and maps processor-specific signals to the monitoring interface. The Event Collector identifies relevant execution events, while the Event Counter Bank accumulates the corresponding event counts. The Analytics Engine calculates derived metrics, including IPC, CPI, pipeline efficiency, stage utilization, hazard distribution, memory operation ratio, and forwarding efficiency. The Mode Controller selects the required analysis mode, and the Register Bank stores the resulting statistics. The supplied project paper reports evaluation using a 32-bit, three-stage pipelined RV32I processor with approximately 2K- and 10K-instruction workloads. The reported results include IPC of 0.92, CPI of 1.07–1.08, approximately 92% pipeline efficiency, and 100% forwarding efficiency under the implemented counting conventions. The results illustrate how the proposed framework can help designers examine instruction throughput, pipeline activity, and the occurrence of hazards without depending entirely on manual waveform inspection. The project also demonstrates the importance of modular RTL design. By separating processor-specific signal mapping from event collection and metric calculation, the analytics architecture can potentially be adapted to other compatible RISC-V processor implementations. The work is evaluated entirely through digital design and simulation. Physical hardware implementation is outside the scope of the project. Overall, the proposed analytics IP provides a reusable foundation for automated processor-performance evaluation and supports systematic investigation of pipeline behavior during RTL verification and computer architecture development.

11.2 FUTURE SCOPE

11.2.1 Support for additional RISC-V processors

The wrapper interface can be adapted to processors with different pipeline depths and organizations. Each adaptation should include verification of event definitions and stage-specific metrics.

11.2.2 Additional performance counters

The design can be extended with counters for other implementation-relevant events, depending on the signals available in the target processor. Examples may include cache misses, branch-prediction events, exception activity, and memory-latency indicators.

11.2.3 Improved arithmetic precision

Future versions can use wider intermediate registers or fixed-point arithmetic to improve the precision of calculated metrics. Explicit handling of division by zero and overflow can further improve result reliability.

11.2.4 Automated result export

The simulation environment can be extended to export counter values and derived metrics into structured text files, CSV files, or other machine-readable formats. This would simplify the comparison of multiple workloads.

11.2.5 Automated verification

Assertions and self-checking testbenches can be introduced to verify counter-update conditions, mode transitions, reset behavior, and metric calculations automatically.

11.2.6 Performance visualization

A software-based visualization tool can be developed to display IPC, CPI, hazard distribution, forwarding efficiency, and pipeline-stage utilization in graphical form.

11.2.7 Synthesis and implementation evaluation

A future study could synthesize the analytics IP and evaluate its area, timing, and estimated power using an appropriate digital design flow. Such results would be needed to assess the implementation cost of adding the analytics logic to a processor.

CHAPTER 12
REFERENCES

[1] RISC-V International, “The RISC-V Instruction Set Manual, Volume II: Privileged Architecture,” RISC-V International, 2026.

[2] D. Schiavone, Ed., “Performance Counters — CORE-V CV32E41P User Manual,” OpenHW Group, 2020.

[3] Linux perf wiki Contributors, “perf: Linux profiling with performance counters,” Linux perf Wiki, 2024.

[4] E. Karabulut and A. Aysu, “Highly Reconfigurable Performance Monitoring Unit on RISC-V,” in Proc. 2020 IEEE 15th International Conference on Solid-State & Integrated Circuit Technology (ICSICT), pp. 1–3, 2020, doi: 10.1109/ICSICT49897.2020.9278263, 2020.

[5] F. Cosimi, F. Tronci, S. Saponara, and P. Gai, “Analysis, Hardware Specification and Design of a Programmable Performance Monitoring Unit (PPMU) for RISC-V ECUs,” in Proc. 2022 IEEE International Conference on Smart Computing (SMARTCOMP), pp. 213–218, 2022, doi: 10.1109/SMARTCOMP55677.2022.00056, 2022.

[6] J. M. Domingos, T. Rocha, N. Neves, N. Roma, P. Tomás, and L. Sousa, “Supporting RISC-V performance counters through Linux performance analysis tools,” in Proc. 2023 IEEE 34th International Conference on Application-Specific Systems, Architectures and Processors (ASAP), pp. 94–101, doi: 10.1109/ASAP57973.2023.0002, 2023.

[7] A. Batashev, “Dissecting RISC-V performance: Practical PMU profiling and hardware-agnostic roofline analysis on emerging platforms,” in Parallel Computing Technologies: 18th International Conference, PaCT 2025, Proceedings, Lecture Notes in Computer Science, vol. 16185, pp. 153–168, Springer, doi: 10.1007/978-3-032-06751-7_11, 2026.

[8] T. Gokulan, A. Muraleedharan, and K. Varghese, “Design of a 32-bit, dual pipeline superscalar RISC-V processor on FPGA,” in Proc. 2020 23rd Euromicro Conference on Digital System Design (DSD), pp. 340–343, doi: 10.1109/DSD51259.2020.00062, 2020.

[9] F. Banchelli, R. A. B. Esqueu, T. Rocha, N. Roma, P. Tomás, N. Neves, and F. Mantovani, “RISC-V in HPC: A Look Into Tools for Performance Monitoring,” in High Performance Computing—ISC High Performance 2025 International Workshops, Revised Selected Papers, Lecture Notes in Computer Science, Springer, pp. 562–575, doi: 10.1007/978-3-032-07612-0_43, 2026.

[10] A. Yasin, “A top-down method for performance analysis and counters architecture,” in Proc. 2014 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), pp.35–44,doi:10.1109/ISPASS.2014.6844459,2014.

[11] M. Jiménez Arribas, A. Martínez Hellín, M. Prieto Mateo, I. Gamino del Río, A. Fernández Gallego, Ó. Rodríguez Polo, A. da Silva, P. Parra, and S. Sánchez, “Design and Implementation of a Synchronous Hardware Performance Monitor for a RISC-V Space-Oriented Processor,” Microprocessors and Microsystems, vol. 112, Art. no. 105132, Feb. 2025, doi: 10.1016/j.micpro.2024.105132.

[12] A. Fernández Gallego, M. Jiménez Arribas, I. Gamino del Río, A. Martínez Hellín, M. Prieto Mateo, Ó. Rodríguez Polo, A. da Silva, P. Parra, and S. Sánchez, "Count Overflow and Privilege Mode Filtering Extension Implementation on a RISC-V On-Board Processor," Microprocessors and Microsystems, vol. 109, Art. no. 105084, Sep. 2024, doi: 10.1016/j.micpro.2024.105084.

[13] RISC-V International, "RISC-V Supervisor Binary Interface Specification, Version 3.0: Performance Monitoring Unit Extension," RISC-V International, 2025. [Online]. Available: https://docs.riscv.org/reference/sbi/v3.0/index.html.

[14] OpenHW Group, "CSR Performance Counters — CVA6 User Manual," OpenHW Group. [Online]. Available: https://github.com/openhwgroup/cva6/blob/master/docs/01_cva6_user/CSR_Performance_Counters.rst.

[15] lowRISC, "Performance Counters — Ibex Documentation," lowRISC. [Online]. Available: https://ibex-core.readthedocs.io/en/latest/03_reference/performance_counters.html.

CHAPTER 13
APPENDICES

13.1 PERFORMANCE METRIC DEFINITIONS AND CALCULATION EXAMPLES

This appendix presents the principal equations used by the Performance Monitoring and Analytics IP.

A. Instructions Per Cycle (IPC)

For the reported 10K-instruction workload:

$$= \frac{10144}{10915} \approx 0.929$$

The value is approximately 0.93 when calculated directly from the reported raw counts. The project paper reports IPC as 0.92, so the precise calculation and reporting convention should be checked against the RTL implementation.

B. Cycles Per Instruction (CPI)

For the reported 10K-instruction workload:

$$= \frac{10915}{10144} \approx 1.076$$

The result is approximately 1.08, consistent with the reported CPI after rounding.

C. Forwarding efficiency

For the reported 10K workload:

$$= \frac{1253}{1253} \times 100\% = 100\%$$

D. Stage utilization

For the 10K workload, the reported instruction-fetch-stage utilization is approximately:

$$= \frac{9968}{10915} \times 100\% \approx 91.32\%$$

This agrees with the reported value of approximately 91%.

These calculations illustrate how raw counter values can be converted into performance indicators. Any discrepancy between an independently calculated result and a reported value should be investigated using the actual RTL and the metric definitions.

13.2 COMPARATIVE PERFORMANCE RESULTS

The project evaluates two workloads to compare processor behavior under different instruction-program sizes. The 10K workload reports 10,915 cycles and 10,144 retired instructions, while the 2K workload reports 2,198 cycles and 2,031 retired instructions. The reported IPC values are approximately 0.92 for both workloads. The CPI values are 1.07 and 1.08, respectively. Pipeline efficiency is reported as 92%, and forwarding efficiency is reported as 100% for both cases. The comparison illustrates the value of maintaining a consistent set of performance counters across multiple experiments. The analysis also highlights the importance of validating derived metrics against the raw counters. The final report should document the exact formulas, arithmetic precision, and rounding rules used in the RTL implementation.

Annexure - I

PROJECT TITLE	**A CONFIGURABLE PERFORMANCE MONITORING AND ANALYTICS IP FOR RISC-V PROCESSORS**
PROGRAM	B.E. ECE
STUDENT NAME	LAKSANA , PRANAV J, PRAVIN A, THRISHNA M
REGISTRATION NUMBER	722824106077; 722824106117; 722824106121; 722824106175
NAME OF THE SUPERVISOR AND DESIGNATION	Dr. M. Saravanan, M.E., Ph.D., Assistant Professor/ECE
NAME OF THE SDG GOALS MAPPED	• SDG 9 – Industry, Innovation and Infrastructure • SDG 4 – Quality Education.
SDG GOAL NUMBER	SDG 9, SDG 4
NAME OF THE TRL LEVEL	Experimental proof of concept (technology validated in a laboratory environment)
MENTION THE TRL LEVEL	TRL 4

POs & PSOs Mapping (Put a tick mark in the mapped PO's & PSO's):

Program Outcomes											Program Specific Outcomes	
PO1	**PO2**	**PO3**	**PO4**	**PO5**	**PO6**	**PO7**	**PO8**	**PO9**	**PO10**	**PO11**	**PSO1**	**PSO2**
✓	✓	✓	✓	✓			✓	✓		✓	✓	

Signature of the Supervisor
(Dr. M. Saravanan, M.E., Ph.D.)